ThinkTemplate
开发手册

目　录

介绍
安装配置
模板渲染
变量输出
使用函数
运算符
原样输出
模板注释
模板布局
模板继承
包含文件
输出替换
标签库
内置标签
- 循环标签
- 比较标签
- 条件判断
- 资源文件
- 标签嵌套
- 原生PHP
- 定义标签

标签扩展

介绍

> ThinkTemplate 是一个基于XML的性能卓越的编译型模板引擎，支持两种类型的模板标签，使用了动态编译和缓存技术，而且支持自定义标签库，一直作为 ThinkPHP 的内置模板引擎，现已经支持独立使用。

主要特性：

- 支持模板标签定界符定义；
- 支持直接使用PHP代码书写；
- 支持文件包含；
- 支持多级标签嵌套；
- 支持布局模板功能；
- 一次编译多次运行，编译和运行效率非常高；
- 模板文件和布局模板更新，自动更新模板缓存；
- 系统变量无需赋值直接输出；
- 支持多维数组的快速输出；
- 支持模板变量的默认值；
- 支持页面代码去除Html空白；
- 支持变量组合调节器和格式化功能；
- 允许定义模板禁用函数和禁用PHP语法；
- 通过标签库方式扩展。

每个模板文件在执行过程中都会生成一个编译后的缓存文件，其实就是一个可以运行的PHP文件。

> 由于编译型模板引擎的特性，模板缓存不能关闭，就算关闭缓存也会在每次渲染的时候重新生成模板缓存。

模板引擎支持普通标签和XML标签方式两种标签定义，分别用于不同的目的：

标签类型	描述
普通标签	主要用于输出变量、函数过滤和做一些基本的运算操作
XML标签	也称为标签库标签，主要完成一些逻辑判断、控制和循环输出，并且可扩展

这种方式的结合保证了模板引擎的简洁和强大的有效融合。

模板文件可以同时包含普通标签和标签库标签，标签的定界符都可以重新配置。

普通标签

普通标签用于变量输出和模板注释，普通模板标签默认以 `{` 和 `}` 作为开始和结束标识，并且在开始标记紧跟标签的定义，如果之间有空格或者换行则被视为非模板标签直接输出。 例如： `{$name}` 、 `{$vo.name}` 、 `{$vo['name']|strtoupper}` 都属于正确的标签，而 `{ $name}` 、 `{ $vo.name}` 则不属于。

要更改普通标签的起始标签和结束标签，可以更改 `template.php` 中的配置参数：

```
// 普通标签开始标记
    'tpl_begin' =>    '<{',
    // 普通标签结束标记
    'tpl_end'   =>    '}>'
```

普通标签的定界符就被修改了，原来的 `{$name}` 和 `{$vo.name}` 必须使用 `<{$name}>` 和 `<{$vo.name}>` 才能生效了。

> 本手册后面的内容均使用默认的标签定界符配置进行说明

标签库标签

标签库标签可以用于模板变量输出、文件包含、条件控制、循环输出等功能，而且完全可以自己扩展功能。5.1版本的标签库默认定界符和普通标签一样使用 `{` 和 `}` ，是为了便于在编辑器里面编辑不至于报错，当然，你仍然可以更改标签库标签的起始和结束标签，修改下面的配置参数：

```
//标签库标签开始标签
    'taglib_begin'  =>  '<',
    //标签库标签结束标记
    'taglib_end'    =>  '>',
```

原来的

```
{eq name="name" value="value"}
相等
{else/}
不相等
{/eq}
```

就需要改成

```
<eq name="name" value="value">
相等
<else/>
不相等
```

```
</eq>
```

安装配置

安装

通过 `composer` 安装即可

```
composer require topthink/think-template
```

配置

可以在实例化 `think\Template` 类的时候传入模板引擎的配置参数

```
// 设置模板引擎参数
$config = [
    'view_path'     =>  './template/',
    'cache_path'    =>  './runtime/',
    'view_suffix'   =>  'html',
];

$template = new \think\Template($config);
```

在实例化之后也可以调用 `config` 方法完成配置

```
$config = [
    'view_path'     =>  './template/',
    'cache_path'    =>  './runtime/',
    'view_suffix'   =>  'html',
];

$template = new \think\Template();
$template->config($config);
```

如果在ThinkPHP `6.0+` 中使用的话，只需要在 `view.php` 配置文件中进行模板配置定义即可。

```
// 模板引擎配置
return [
    // 模板引擎类型使用ThinkTemplate
    'type'          => 'Think',
    // 默认模板渲染规则 1 解析为小写+下划线 2 全部转换小写 3 保持操作方法
    'auto_rule'     => 1,
    // 模板目录名
    'view_dir_name' => 'view',
    // 模板后缀
```

```
    'view_suffix'   => 'html',
];
```

系统会自动定位模板目录，你无需配置 `view_path` 参数。

配置参数

支持如下配置参数：

参数名	默认值	描述
view_path	空	模板路径
view_suffix	'html'	默认模板文件后缀
view_depr	DIRECTORY_SEPARATOR	模板文件分隔符
cache_path	空	模板缓存目录
cache_suffix	'php'	默认模板缓存后缀
tpl_deny_func_list	'echo,exit'	模板引擎禁用函数
tpl_deny_php	false	默认模板引擎是否禁用PHP原生代码
tpl_begin	'{'	模板引擎普通标签开始标记
tpl_end	'}'	模板引擎普通标签结束标记
strip_space	false	是否去除模板文件里面的html空格与换行
tpl_cache	true	是否开启模板编译缓存,设为false则每次都会重新编译
compile_type	'file'	模板编译类型
cache_prefix	''	模板缓存前缀标识，可以动态改变
cache_time	0	模板缓存有效期 0 为永久，(以数字为值，单位:秒)
layout_on	false	布局模板开关
layout_name	'layout'	布局模板入口文件
layout_item	'{__CONTENT__}'	布局模板的内容替换标识
taglib_begin	'{'	标签库标签开始标记

taglib_end	'}'	标签库标签结束标记
taglib_load	true	是否使用内置标签库之外的其它标签库，默认自动检测
taglib_build_in	'cx'	内置标签库名称(标签使用不必指定标签库名称),以逗号分隔 注意解析顺序
taglib_pre_load	''	需要额外加载的标签库(须指定标签库名称)，多个以逗号分隔
display_cache	false	模板渲染缓存
cache_id	''	模板缓存ID
tpl_replace_string	[]	
tpl_var_identify	'array'	.语法变量识别，array
default_filter	'htmlentities'	默认过滤方法 用于普通标签输出

模板渲染

模板渲染

模板渲染首先需要设置模板基础路径 `view_path` ，然后直接使用 `fetch` 方法进行模板渲染输出，例如：

```
$template->fetch('Test/Hello');
```

表示系统会按照默认规则自动定位模板文件，其规则是：

```
view_path/Test/Hello.html
```

模板文件严格区分大小写，模板文件的后缀由 `view_suffix` 参数配置，默认为 `html` 。
如果有更改模板引擎的 `view_depr` 设置（假设是 `_` ）的话，则上面的自动定位规则变成：

```
view_path/Test_Hello.html
```

支持指定完整的模板文件方式来渲染模板（支持其它格式的文件后缀），例如：

```
$template->fetch('../template/public/menu.html');
```

> 如果是在ThinkPHP `6.0+` 中使用的话，无需设置 `view_path` 参数，会自动定位。并且提供了三种不同的模板文件渲染规则。

在ThinkPHP中进行模板渲染的时候会自动进行规范化处理，默认的模板文件名规则为实际操作方法名的小写+下划线写法。但可以配置 `auto_rule` 参数的值来改变当前操作的自动渲染规则。

auto_rule配置	自动定位规则
1（默认值）	操作方法的小写+下划线
2	操作方法全部转换小写
3	保持和操作方法一致

渲染内容

如果希望直接解析内容而不通过模板文件的话，可以使用 `display` 方法：

```
// 直接渲染内容
$content = '{$name}-{$email}';
$temlate->display($content, ['name' => 'thinkphp', 'email' => 'thinkphp@qq.com']
```

```
);
```

渲染的内容中一样可以使用模板引擎的相关标签。

变量输出

在模板中输出变量的方法很简单，例如，在控制器的方法中我们给模板变量赋值：

```
// 设置模板引擎参数
$config = [
    'view_path'     =>  './template/',
    'cache_path'    =>  './runtime/',
    'view_suffix'   =>  'html',
];

$template = new \think\Template($config);
$template->fetch('hello', ['name' => 'ThinkPHP']);
```

在 `hello.html` 模板文件中使用：

```
Hello,{$name}!
```

模板编译后的结果就是：

```
Hello,<?php echo htmlentities($name);?>!
```

这样，运行的时候就会在模板中显示： `Hello,ThinkPHP!`

注意模板标签的 `{` 和 `$` 之间不能有任何的空格，否则标签无效。所以，下面的标签

```
Hello,{ $name}!
```

将不会正常输出name变量，而是直接保持不变输出： `Hello,{ $name}!`

模板标签的变量输出根据变量类型有所区别，刚才我们输出的是字符串变量，如果是数组变量，

```
$data['name'] = 'ThinkPHP';
$data['email'] = 'thinkphp@qq.com';
$template->fetch('hello', ['data' => $data]);
```

那么，在模板中我们可以用下面的方式输出：

```
Name：{$data.name}
Email：{$data.email}
```

或者用下面的方式也是有效：

```
Name：{$data['name']}
Email：{$data['email']}
```

> 当我们要输出多维数组的时候，往往要采用后面一种方式。

如果 `data` 变量是一个对象（并且包含有 `name` 和 `email` 两个属性），那么可以用下面的方式输出：

```
Name：{$data->name}
Email：{$data->email}
```

也可以直接调用对象的常量或者方法

```
常量：{$data::CONST_NAME}
方法：{$data->fun()}
```

使用默认值

我们可以给变量输出提供默认值，例如：

```
{$user.nickname|default="这家伙很懒，什么也没留下"}
```

系统变量输出

普通的模板变量需要首先赋值后才能在模板中输出，但是系统变量则不需要，可以直接在模板中输出，系统变量的输出通常以 `{$Request.` （大小写一致）打头，例如：

```
{$Request.server.script_name} // 输出$_SERVER['SCRIPT_NAME']变量
{$Request.session.user_id} // 输出$_SESSION['user_id']变量
{$Request.get.page} // 输出$_GET['page']变量
{$Request.cookie.name}  // 输出$_COOKIE['name']变量
```

支持输出 `$_SERVER` 、 `$_ENV` 、 `$_POST` 、 `$_GET` 、 `$_REQUEST` 、 `$_SESSION` 和 `$_COOKIE` 变量。

如果在ThinkPHP6.0中使用的话，模板还支持直接输出 `Request` 请求对象的方法，用法如下：

> $Request.方法名.参数

例如：

```
{$Request.param.name}
```

> 支持 `Request` 类的大部分方法，但只支持方法的第一个参数。

下面都是有效的输出：

```
// 调用Request对象的param方法 传入参数为name
{$Request.param.name}
// 调用Request对象的param方法 传入参数为user.nickname
{$Request.param.user.nickname}
// 调用Request对象的root方法
{$Request.root}
// 调用Request对象的root方法，并且传入参数true
{$Request.root.true}
// 调用Request对象的path方法
{$Request.path}
// 调用Request对象的module方法
{$Request.module}
// 调用Request对象的controller方法
{$Request.controller}
// 调用Request对象的action方法
{$Request.action}
// 调用Request对象的ext方法
{$Request.ext}
// 调用Request对象的host方法
{$Request.host}
// 调用Request对象的ip方法
{$Request.ip}
// 调用Request对象的header方法
{$Request.header.accept-encoding}
```

常量输出

还可以输出常量

```
{$Think.const.PHP_VERSION}
```

或者直接使用

```
{$Think.PHP_VERSION}
```

配置输出

仅用于输出ThinkPHP `6.0+` 中的配置参数使用：

```
{$Think.config.app.app_host}
{$Think.config.session.name}
```

语言变量

仅用于输出ThinkPHP `6.0+` 的语言变量使用：

```
{$Think.lang.page_error}
{$Think.lang.var_error}
```

扩展解析

可以自己扩展特殊的变量标签解析，使用

```
$template->extend('$Cms', function(array $vars) {
    return implode('.', $vars);
});
```

例如使用（注意严格区分大小写）

```
{$Cms.test.name}
```

会输出

```
test.name
```

使用函数

需要对模板输出使用函数进行过滤或其它处理的时候，可以使用：

```
{$data.name|md5}
```

可以使用空格，例如下面的写法是一样的：

```
{$data.name | md5}
```

编译后的结果是：

```
<?php echo htmlentities(md5($data['name'])); ?>
```

其中 `htmlentities` 方法是系统默认添加的（无需手动指定。

> 为了避免出现XSS安全问题，默认的变量输出都会使用 `htmlentities` 方法进行转义输出。

你还可以设置默认的过滤方法，在配置文件 `template.php` 中设置

```
'default_filter' => 'htmlspecialchars'
```

就会默认使用 `htmlspecialchars` 方法过滤输出。
如果你不需要转义（例如你需要输出html表格等内容），可以使用：

```
{$data.name|raw}
```

编译后的结果是：

```
<?php echo $data['name']; ?>
```

系统内置了下面几个固定的过滤规则（不区分大小写）

过滤方法	描述
date	日期格式化（支持各种时间类型）
format	字符串格式化
upper	转换为大写

lower	转换为小写
first	输出数组的第一个元素
last	输出数组的最后一个元素
default	默认值
raw	不使用（默认）转义

例如

```
{$data.create_time|date='Y-m-d H:i'}
{$data.number|format='%02d'}
```

如果函数有多个参数需要调用，可以使用

```
{$data.name|substr=0,3}
```

表示输出

```
<?php echo htmlentities(substr($data['name'],0,3)); ?>
```

还可以支持多个函数过滤，多个函数之间用“|”分割即可，例如：

```
{$name|md5|upper|substr=0,3}
```

编译后的结果是：

```
<?php echo htmlentities(substr(strtoupper(md5($name)),0,3)); ?>
```

函数会按照从左到右的顺序依次调用（系统默认的过滤规则会在最后调用）。

> 变量输出使用的函数可以支持内置的PHP函数或者用户自定义函数，甚至是静态方法。

如果你觉得这样写起来比较麻烦，也可以直接这样写：

```
{:substr(strtoupper(md5($name)),0,3)}
```

> 使用该方法输出的值不会使用默认的过滤方法进行转义。

可以在模板中直接使用系统的助手函数进行输出

```
{:app('cache')->get('name')}
```

表示调用容器中的 `think\Cache` 对象实例输出 `name` 缓存标识内容。

`{:` 开头的变量输出表示调用函数或者类的方法及属性，如果你要带命名空间调用类的属性，例如：

```
{:think\\App::VERSION}
{:think\\facade\\Request::get('name')}
```

> 类的命名空间中的 `\` 需要改成 `\\` 才能正常调用。

运算符

我们可以对模板输出使用运算符，包括如下支持。

运算符	使用示例
`+`	`{$a+$b}`
`-`	`{$a-$b}`
`*`	`{$a*$b}`
`/`	`{$a/$b}`
`%`	`{$a%$b}`
`++`	`{$a++}` 或 `{++$a}`
`--`	`{$a--}` 或 `{--$a}`
综合运算	`{$a+$b*10+$c}`

在使用运算符的时候，不再支持前面提到的函数过滤用法，例如：

```
{$user.score+10} //正确的
{$user['score']+10} //正确的
{$user['score']*$user['level']} //正确的
{$user['score']|myFun*10} //错误的
{$user['score']+myFun($user['level'])} //正确的
```

三元运算

模板可以支持三元运算符，例如：

```
{$status? '正常' : '错误'}
{$info['status']? $info['msg'] : $info['error']}
{$info.status? $info.msg : $info.error }
```

还支持如下的写法：

```
{$name ?? '默认值'}
```

表示如果有设置 `$name` 则输出 `$name` ,否则输出 `默认值` 。

```
{$name?='默认值'}
```

表示$name为真时才输出默认值。

```
{$name ?: 'NO'}
```

表示如果$name为真则输出$name，否则输出NO。

```
{$a==$b ? 'yes' : 'no'}
```

前面的表达式为真输出yes,否则输出no， 条件可以是==、===、!=、!==、>=、<=

原样输出

可以使用 `literal` 标签来防止模板标签被解析，例如：

```
{literal}

Hello,{$name}!

{/literal}
```

上面的 `{$name}` 标签被 `literal` 标签包含，因此并不会被模板引擎解析，而是保持原样输出。

`literal` 标签还可以用于页面的JS代码外层，确保JS代码中的某些用法和模板引擎不产生混淆。
总之，所有可能和内置模板引擎的解析规则冲突的地方都可以使用 `literal` 标签处理。

模板注释

模板支持注释功能，该注释文字在最终页面不会显示，仅供模板制作人员参考和识别。

单行注释

格式：

```
{/\* 注释内容 \*/ } 或 {// 注释内容 }
```

例如：

```
{// 这是模板注释内容 }
```

> 注意 `{` 和注释标记之间不能有空格。

多行注释

支持多行注释，例如：

```
{/* 这是模板
注释内容*/ }
```

模板注释支持多行，模板注释在生成编译缓存文件后会自动删除，这一点和Html的注释不同。

模板布局

ThinkPHP的模板引擎内置了布局模板功能支持，可以方便的实现模板布局以及布局嵌套功能。

有三种布局模板的支持方式：

第一种方式：全局配置方式

这种方式仅需在项目配置文件中添加相关的布局模板配置，就可以简单实现模板布局功能，比较适用于全站使用相同布局的情况，需要配置开启 `layout_on` 参数（默认不开启），并且设置布局入口文件名 `layout_name` （默认为layout）。

```
return  [
    'layout_on'     =>  true,
    'layout_name'   =>  'layout',
]
```

开启 `layout_on` 后，我们的模板渲染流程就有所变化，例如：

```
namespace app\index\controller;

use think\Controller;

Class User extends Controller
{
     public function add()
     {
         return $this->fetch('add');
     }
}
```

在不开启 `layout_on` 布局模板之前，会直接渲染 `application/index/view/user/add.html` 模板文件,开启之后，首先会渲染 `application/index/view/layout.html` 模板，布局模板的写法和其他模板的写法类似，本身也可以支持所有的模板标签以及包含文件，区别在于有一个特定的输出替换变量 `{__CONTENT__}` ，例如，下面是一个典型的layout.html模板的写法：

```
{include file="public/header" /}
 {__CONTENT__}
{include file="public/footer" /}
```

读取layout模板之后，会再解析 `user/add.html` 模板文件，并把解析后的内容替换到layout布局模板文件

的 `{CONTENT}` 特定字符串。

当然可以通过设置来改变这个特定的替换字符串，例如：

```
return [
    'layout_on'     =>  true,
    'layout_name'   =>  'layout',
    'layout_item'   =>  '{__REPLACE__}'
]
```

> 一个布局模板同时只能有一个特定替换字符串。
>
> 采用这种布局方式的情况下，一旦user/add.html 模板文件或者layout.html布局模板文件发生修改，都会导致模板重新编译。
>
> 如果需要指定其他位置的布局模板，可以使用：

```
return [
    'layout_on'     =>  true,
    'layout_name'   =>  'layout/layoutname',
    'layout_item'   =>  '{__REPLACE__}'
]
```

就表示采用 `application/index/view/layout/layoutname.html` 作为布局模板。

如果某些页面不需要使用布局模板功能，可以在模板文件开头加上 `{__NOLAYOUT__}` 字符串。

如果上面的user/add.html 模板文件里面包含有 `{__NOLAYOUT__}` ，则即使当前开启布局模板，也不会进行布局模板解析。

第二种方式：模板标签方式

这种布局模板不需要在配置文件中设置任何参数，也不需要开启 `layout_on` ，直接在模板文件中指定布局模板即可，相关的布局模板调整也在模板中进行。

以前面的输出模板为例，这种方式的入口还是在user/add.html 模板，但是我们可以修改下add模板文件的内容，在头部增加下面的布局标签（记得首先关闭前面的 `layout_on` 设置，否则可能出现布局循环）：

```
{layout name="layout" /}
```

表示当前模板文件需要使用 `layout.html` 布局模板文件，而布局模板文件的写法和上面第一种方式是一样的。当渲染 `user/add.html` 模板文件的时候，如果读取到layout标签，则会把当前模板的解析内容替换到layout布局模板的 `{CONTENT}` 特定字符串。

一个模板文件中只能使用一个布局模板，如果模板文件中没有使用任何layout标签则表示当前模板不使用任何布局。

如果需要使用其他的布局模板，可以改变layout的name属性，例如：

```
{layout name="newlayout" /}
```

还可以在layout标签里面指定要替换的特定字符串：

```
{layout name="Layout/newlayout" replace="[__REPLACE__]" /}
```

第三种方式：动态方法布局

使用内置的layout方法可以更灵活的在程序中控制模板输出的布局功能，尤其适用于局部需要布局或者关闭布局的情况，这种方式也不需要在配置文件中开启 `layout_on` 。例如：

```
namespace app\index\controller;

use think\Controller;

class User extends Controller
{
     public function add()
     {
         $this->view->engine->layout(true);
         return $this->fetch('add');
     }
}
```

表示当前的模板输出启用了布局模板，并且采用默认的layout布局模板。

如果当前输出需要使用不同的布局模板，可以动态的指定布局模板名称，例如：

```
namespace app\index\controller;

use think\Controller;
```

```
class User extends Controller
{
     public function add()
     {
         $this->view->engine->layout('Layout/newlayout');
         return $this->display('add');
     }
}
```

或者使用layout方法动态关闭当前模板的布局功能（这种用法可以配合第一种布局方式，例如全局配置已经开启了布局，可以在某个页面单独关闭）：

```
namespace app\index\controller;

use think\Controller;

class User extends Controller
{
     public function add()
     {
        // 临时关闭当前模板的布局功能
         $this->view->engine->layout(false);
         return $this->display('add');
     }
}
```

> 三种模板布局方式中，第一种和第三种是在程序中配置实现模板布局，第二种方式则是单纯通过模板标签在模板中使用布局。具体选择什么方式，需要根据项目的实际情况来了。

模板继承

模板继承是一项更加灵活的模板布局方式，模板继承不同于模板布局，甚至来说，应该在模板布局的上层。模板继承其实并不难理解，就好比类的继承一样，模板也可以定义一个基础模板（或者是布局），并且其中定义相关的区块（block），然后继承（extend）该基础模板的子模板中就可以对基础模板中定义的区块进行重载。

因此，模板继承的优势其实是设计基础模板中的区块（block）和子模板中替换这些区块。

每个区块由 `{block} {/block}` 标签组成。 下面就是基础模板中的一个典型的区块设计（用于设计网站标题）：

```
{block name="title"}<title>网站标题</title>{/block}
```

block标签必须指定name属性来标识当前区块的名称，这个标识在当前模板中应该是唯一的，block标签中可以包含任何模板内容，包括其他标签和变量，例如：

```
{block name="title"}<title>{$web_title}</title>{/block}
```

你甚至还可以在区块中加载外部文件：

```
{block name="include"}{include file="Public:header" /}{/block}
```

一个模板中可以定义任意多个名称标识不重复的区块，例如下面定义了一个 `base.html` 基础模板：

```
<html>
<head>
<meta http-equiv="Content-Type" content="text/html; charset=utf-8">
<title>{block name="title"}标题{/block}</title>
</head>
<body>
{block name="menu"}菜单{/block}
{block name="left"}左边分栏{/block}
{block name="main"}主内容{/block}
{block name="right"}右边分栏{/block}
{block name="footer"}底部{/block}
</body>
</html>
```

然后我们在子模板（其实是当前操作的入口模板）中使用继承：

```
{extend name="base" /}

{block name="title"}{$title}{/block}

{block name="menu"}
<a href="/" >首页</a>
<a href="/info/" >资讯</a>
<a href="/bbs/" >论坛</a>
{/block}

{block name="left"}{/block}

{block name="main"}
{volist name="list" id="vo"}
<a href="/new/{$vo.id}">{$vo.title}</a><br/>
 {$vo.content}
{/volist}
{/block}

{block name="right"}
 最新资讯：
{volist name="news" id="new"}
<a href="/new/{$new.id}">{$new.title}</a><br/>
{/volist}
{/block}

{block name="footer"}
{__block__}
 @ThinkPHP 版权所有
{/block}
```

上例中，我们可以看到在子模板中使用了extend标签来继承了base模板。

在子模板中，可以对基础模板中的区块进行重载定义，如果没有重新定义的话，则表示沿用基础模板中的区块定义，如果定义了一个空的区块，则表示删除基础模板中的该区块内容。 上面的例子，我们就把left区块的内容删除了，其他的区块都进行了重载。而

```
{block name="footer"}
{__block__}@ThinkPHP 版权所有
{/block}
```

这一区块中有 `{__block__}` 这个标签，当区块中有这个标记时，就不只是直接重载这个区块，它表示引用所继承模板对应区块的内容到这个位置，最终这个区块是合并后的内容。所以这里footer区块最后的内容是： 底部 @ThinkPHP 版权所有

extend标签的用法和include标签一样，你也可以加载其他模板：

```
{extend name="Public:base" /}
```

或者使用绝对文件路径加载

```
{extend name="./Template/Public/base.html" /}
```

在当前子模板中，只能定义区块而不能定义其他的模板内容，否则将会直接忽略，并且只能定义基础模板中已经定义的区块。

例如，如果采用下面的定义：

```
{block name="title"}<title>{$title}</title>{/block}
<a href="/" >首页</a>
<a href="/info/" >资讯</a>
<a href="/bbs/" >论坛</a>
```

导航部分将是无效的，不会显示在模板中。

模板可以多级继承，比如B继承了A，而C又继承了B，最终C中的区块会覆盖B和A中的同名区块，但C和B中的区块必须是A中已定义过的。

子模板中的区块定义顺序是随意的，模板继承的用法关键在于基础模板如何布局和设计规划了，如果结合原来的布局功能，则会更加灵活。

包含文件

在当前模版文件中包含其他的模版文件使用include标签，标签用法：

```
{include file='模版文件1,模版文件2,...' /}
```

> 包含的模板文件中不能再使用模板布局或者模板继承。

使用模版表达式

模版表达式的定义规则为：模块@控制器/操作

例如：

```
{include file="public/header" /} // 包含头部模版header
{include file="public/menu" /} // 包含菜单模版menu
{include file="blue/public/menu" /} // 包含blue主题下面的menu模版
```

可以一次包含多个模版，例如：

```
{include file="public/header,public/menu" /}
```

> 注意，包含模版文件并不会自动调用控制器的方法，也就是说包含的其他模版文件中的变量赋值需要在当前操作中完成。

使用模版文件

可以直接包含一个模版文件名（包含完整路径），例如：

```
{include file="../application/view/default/public/header.html" /}
```

> 路径以 项目目录/public/ 路径下为起点

传入参数

无论你使用什么方式包含外部模板，Include标签支持在包含文件的同时传入参数，例如，下面的例子我们在包含header模板的时候传入了 `title` 和 `keywords` 参数：

```
{include file="Public/header" title="$title" keywords="开源WEB开发框架" /}
```

就可以在包含的header.html文件里面使用title和keywords变量，如下：

```
<html xmlns="http://www.w3.org/1999/xhtml">
<head>
<title>[title]</title>
<meta name="keywords" content="[keywords]" />
</head>
```

上面title参数传入的是个变量 `$title` ，模板内的 `[title]` 最终会替换成 `$title` 的值，当然 `$title` 这个变量必须要存在。

> 包含文件中可以再使用include标签包含别的文件，但注意不要形成A包含A，或者A包含B而B又包含A这样的死循环。
>
> [danger] 由于模板解析的特点，从入口模板开始解析，如果外部模板有所更改，模板引擎并不会重新编译模板，除非在调试模式下或者缓存已经过期。如果部署模式下修改了包含的外部模板文件后，需要把模块的缓存目录清空，否则无法生效。

输出替换

模板输出替换

支持对模板文件输出的内容进行字符替换，定义后在渲染模板或者内容输出的时候就会自动根据设置的替换规则自动替换。

如果需要全局替换的话，可以直接在配置文件中添加：

```
'tpl_replace_string'  =>  [
    '__STATIC__'=>'/static',
    '__JS__' => '/static/javascript',
]
```

> 替换规则严格区分大小写

标签库

内置的模板引擎除了支持普通变量的输出之外，更强大的地方在于标签库功能。

> 标签库类似于Java的Struts中的JSP标签库，每一个标签库是一个独立的标签库文件，标签库中的每一个标签完成某个功能，采用XML标签方式（包括开放标签和闭合标签）。
>
> 标签库分为内置和扩展标签库，内置标签库是 `Cx` 标签库。

导入标签库

使用taglib标签导入当前模板中需要使用的标签库，例如：

```
{taglib name="html" /}
```

> 如果没有定义html标签库的话，则导入无效。
>
> 也可以导入多个标签库，使用：

```
{taglib name="html,article" /}
```

导入标签库后，就可以使用标签库中定义的标签了，假设article标签库中定义了read标签：

```
{article:read name="hello" id="data" }
{$data.id}:{$data.title}
{/article:read}
```

在上面的标签中， `{article:read}... {/article:read}` 就是闭合标签，起始和结束标签必须成对出现。

如果是 `{article:read name="hello" /}` 就是开放标签。

> 闭合和开放标签取决于标签库中的定义，一旦定义后就不能混淆使用，否则就会出现错误。

内置标签

内置标签库无需导入即可使用，并且不需要加XML中的标签库前缀，ThinkPHP内置的标签库是Cx标签库，所以，Cx标签库中的所有标签，我们可以在模板文件中直接使用，我们可以这样使用：

```
{eq name="status" value="1" }
正常
{/eq}
```

如果Cx不是内置标签的话，可能就需要这么使用了：

```
{cx:eq name="status" value="1" }
正常
{/cx:eq}
```

更多的Cx标签库中的标签用法，参考后面的内置标签。

内置标签库可以简化模板中标签的使用，所以，我们还可以把其他的标签库定义为内置标签库（前提是多个标签库没有标签冲突的情况），例如：

```
'taglib_build_in'    =>    'cx,article'
```

配置后，上面的标签用法就可以改为：

```
{read name="hello" id="data" }
{$data.id}:{$data.title}
{/read}
```

标签库预加载

标签库预加载是指无需手动在模板文件中导入标签库即可使用标签库中的标签，通常用于某个标签库需要被大多数模板使用的情况。

在应用或者模块的配置文件中添加：

```
'taglib_pre_load'    =>    'article,html'
```

设置后，模板文件就不再需要使用

```
{taglib name="html,article" /}
```

但是仍然可以在模板中调用：

```
{article:read name="hello" id="data" }
```

```
{$data.id}:{$data.title}
{/article:read}
```

内置标签

变量输出使用普通标签就足够了，但是要完成其他的控制、循环和判断功能，就需要借助模板引擎的标签库功能了，系统内置标签库的所有标签无需引入标签库即可直接使用。

内置标签主要包括：

标签名	作用	包含属性
include	包含外部模板文件（闭合）	file
load	导入资源文件（闭合 包括js css import别名）	file,href,type,value,base path
volist	循环数组数据输出	name,id,offset,length,key,mod
foreach	数组或对象遍历输出	name,item,key
for	For循环数据输出	name,from,to,before,step
switch	分支判断输出	name
case	分支判断输出（必须和switch配套使用）	value,break
default	默认情况输出（闭合 必须和switch配套使用）	无
compare	比较输出（包括eq neq lt gt egt elt heq nheq 等别名）	name,value,type
range	范围判断输出（包括in notin between notbetween别名）	name,value,type
present	判断是否赋值	name
notpresent	判断是否尚未赋值	name
empty	判断数据是否为空	name
notempty	判断数据是否不为空	name
defined	判断常量是否定义	name
notdefined	判断常量是否未定义	name
define	常量定义（闭合）	name,value
assign	变量赋值（闭合）	name,value

if	条件判断输出	condition
elseif	条件判断输出（闭合 必须和if标签配套使用）	condition
else	条件不成立输出（闭合 可用于其他标签）	无
php	使用php代码	无

循环标签

`FOREACH` 标签

`foreach` 标签的用法和PHP语法非常接近，用于循环输出数组或者对象的属性，用法如下：

```
$list = User::all();
View::assign('list',$list);
```

模板文件中可以这样输出

```
{foreach $list as $key=>$vo }
    {$vo.id}:{$vo.name}
{/foreach}
```

可以不通过模板变量赋值，支持使用函数或者方法获取数据循环输出：

```
{foreach :model('user')->all() as $key=>$vo }
    {$vo.id}:{$vo.name}
{/foreach}
```

`VOLIST` 标签

`volist` 标签通常用于查询数据集或者二维数组的结果输出。 在控制器中首先对模版赋值：

```
$list = User::all();
View::assign('list',$list);
```

在模版定义如下，循环输出用户的编号和姓名：

```
{volist name="list" id="vo"}
{$vo.id}:{$vo.name}<br/>
{/volist}
```

`Volist` 标签的 `name` 属性表示模板赋值的变量名称，因此不可随意在模板文件中改变。 `id` 表示当前的循环变量，可以随意指定，但确保不要和 `name` 属性冲突，例如：

```
{volist name="list" id="data"}
{$data.id}:{$data.name}<br/>
{/volist}
```

可以直接使用函数设定数据集，而不需要在控制器中给模板变量赋值传入数据集变量，如：

```
{volist name=":model('user')->all()" id="vo"}
{$vo.name}
{/volist}
```

支持输出查询结果中的部分数据，例如输出其中的第5～15条记录

```
{volist name="list" id="vo" offset="5" length='10'}
{$vo.name}
{/volist}
```

输出偶数记录

```
{volist name="list" id="vo" mod="2" }
{eq name="mod" value="1"}{$vo.name}{/eq}
{/volist}
```

`mod` 属性还用于控制一定记录的换行，例如：

```
{volist name="list" id="vo" mod="5" }
{$vo.name}
{eq name="mod" value="4"}<br/>{/eq}
{/volist}
```

为空的时候输出提示：

```
{volist name="list" id="vo" empty="暂时没有数据" }
{$vo.id}|{$vo.name}
{/volist}
```

`empty` 属性不支持直接传入html语法，但可以支持变量输出，例如：

```
View::assign('empty','<span class="empty">没有数据</span>');
View::assign('list',$list);
```

然后在模板中使用：

```
{volist name="list" id="vo" empty="$empty" }
{$vo.id}|{$vo.name}
```

```
{/volist}
```

输出循环变量：

```
{volist name="list" id="vo" key="k" }
{$k}.{$vo.name}
{/volist}
```

如果没有指定 `key` 属性的话，默认使用循环变量i，例如：

```
{volist name="list" id="vo"  }
{$i}.{$vo.name}
{/volist}
```

如果要输出数组的索引，可以直接使用 `key` 变量，和循环变量不同的是，这个 `key` 是由数据本身决定，而不是循环控制的，例如：

```
{volist name="list" id="vo"  }
{$key}.{$vo.name}
{/volist}
```

`FOR` 标签

用法：

```
{for start="开始值" end="结束值" comparison="" step="步进值" name="循环变量名" }
{/for}
```

开始值、结束值、步进值和循环变量都可以支持变量，开始值和结束值是必须，其他是可选。comparison 的默认值是lt，name的默认值是i，步进值的默认值是1，举例如下：

```
{for start="1" end="100"}
{$i}
{/for}
```

解析后的代码是

```
for ($i=1;$i<100;$i+=1){
    echo $i;
}
```

比较标签

比较标签用于简单的变量比较，复杂的判断条件可以用if标签替换，比较标签是一组标签的集合，基本上用法都一致，如下：

```
{比较标签 name="变量" value="值"}
内容
{/比较标签}
```

系统支持的比较标签以及所表示的含义分别是：

标签	含义
eq或者 equal	等于
neq 或者notequal	不等于
gt	大于
egt	大于等于
lt	小于
elt	小于等于
heq	恒等于
nheq	不恒等于

他们的用法基本是一致的，区别在于判断的条件不同，并且所有的比较标签都可以和else标签一起使用。

例如，要求name变量的值等于value就输出，可以使用：

```
{eq name="name" value="value"}value{/eq}
```

或者

```
{equal name="name" value="value"}value{/equal}
```

也可以支持和else标签混合使用：

```
{eq name="name" value="value"}
相等
{else/}
不相等
{/eq}
```

当 name变量的值大于5就输出

```
{gt name="name" value="5"}value{/gt}
```

当name变量的值不小于5就输出

```
{egt name="name" value="5"}value{/egt}
```

比较标签中的变量可以支持对象的属性或者数组，甚至可以是系统变量，例如： 当vo对象的属性（或者数组，或者自动判断）等于5就输出

```
{eq name="vo.name" value="5"}
{$vo.name}
{/eq}
```

当vo对象的属性等于5就输出

```
{eq name="vo:name" value="5"}
{$vo.name}
{/eq}
```

当$vo['name']等于5就输出

```
{eq name="vo['name']" value="5"}
{$vo.name}
{/eq}
```

而且还可以支持对变量使用函数 当vo对象的属性值的字符串长度等于5就输出

```
{eq name="vo:name|strlen" value="5"}{$vo.name}{/eq}
```

变量名可以支持系统变量的方式，例如：

```
{eq name="Think.get.name" value="value"}相等{else/}不相等{/eq}
```

通常比较标签的值是一个字符串或者数字，如果需要使用变量，只需要在前面添加“$”标志： 当vo对象的属性等于$a就输出

```
{eq name="vo:name" value="$a"}{$vo.name}{/eq}
```

所有的比较标签可以统一使用compare标签（其实所有的比较标签都是compare标签的别名），例如： 当name变量的值等于5就输出

```
{compare name="name" value="5" type="eq"}value{/compare}
```

等效于

```
{eq name="name" value="5" }value{/eq}
```

其中type属性的值就是上面列出的比较标签名称

条件判断

SWITCH标签

用法：

```
{switch 变量 }
    {case value1 }输出内容1{/case}
    {case value2}输出内容2{/case}
    {default /}默认情况
{/switch}
```

使用示例：

```
{switch User.level}
    {case 1}value1{/case}
    {case 2}value2{/case}
    {default /}default
{/switch}
```

可以使用函数以及系统变量，例如：

```
{switch User.level|intval }
    {case 1}admin{/case}
    {default /}default
{/switch}
```

对于case属性可以支持多个条件的判断，使用”|”进行分割，例如：

```
{switch Think.get.type}
    {case gif|png|jpg}图像格式{/case}
    {default /}其他格式
{/switch}
```

表示如果 `$_GET["type"]` 是gif、png或者jpg的话，就判断为图像格式。

也可以对case的value属性使用变量，例如：

```
{switch $User.userId}
    {case $adminId}admin{/case}
    {case $memberId}member{/case}
{/switch}
```

> 使用变量方式的情况下，不再支持 `|` 分割的多个条件判断用法。

IF标签

用法：

```
{if 表达式}value1
{elseif 表达式 /}value2
{else /}value3
{/if}
```

用法示例：

```
{if ( $name == 1) OR ( $name > 100) } value1
{elseif $name == 2 /}value2
{else /} value3
{/if}
```

可以使用php代码，例如：

```
{if strtoupper($user['name']) == 'THINKPHP' }ThinkPHP
{else /} other Framework
{/if}
```

判断条件可以支持点语法和对象语法，例如：

```
{if $user.name == 'ThinkPHP'}ThinkPHP
{else /} other Framework
{/if}

{if $user->name == 'ThinkPHP'}ThinkPHP
{else /} other Framework
{/if}
```

> 如果某些特殊的要求下面，IF标签仍然无法满足要求的话，可以使用原生php代码或者PHP标签来直接书写代码。

范围判断

范围判断标签包括 `in` / `notin` / `between` / `notbetween` 四个标签，都用于判断变量是否中某个范围。

IN和NOTIN

用法： 假设我们中控制器中给id赋值为1：

```
$id =    1;
View::assign('id',$id);
```

我们可以使用in标签来判断模板变量是否在某个范围内，例如：

```
{in name="id" value="1,2,3"}
id在范围内
{/in}
```

最后会输出： `id在范围内` 。

如果判断不在某个范围内，可以使用notin标签：

```
{notin name="id" value="1,2,3"}
id不在范围内
{/notin}
```

最后会输出： `id不在范围内` 。

可以把上面两个标签合并成为：

```
{in name="id" value="1,2,3"}
id在范围内
{else/}
id不在范围内
{/in}
```

name属性还可以支持直接判断系统变量，例如：

```
{in name="Think.get.id" value="1,2,3"}
$_GET['id'] 在范围内
{/in}
```

> 更多的系统变量用法可以参考系统变量部分。

value属性也可以使用变量，例如：

```
{in name="id" value="$range"}
id在范围内
```

```
{/in}
```

$range变量可以是数组，也可以是以逗号分隔的字符串。

value属性还可以使用系统变量，例如：

```
{in name="id" value="$Think.post.ids"}
id在范围内
{/in}
```

BETWEEN 和 NOTBETWEEN

可以使用 `between` 标签来判断变量是否在某个区间范围内，可以使用：

```
{between name="id" value="1,10"}
输出内容1
{/between}
```

同样，也可以使用 `notbetween` 标签来判断变量不在某个范围内：

```
{notbetween name="id" value="1,10"}
输出内容2
{/notbetween}
```

也可以使用 `else` 标签把两个用法合并，例如：

```
{between name="id" value="1,10"}
输出内容1
{else/}
输出内容2
{/between}
```

当使用 `between` 标签的时候， `value` 只需要一个区间范围，也就是只支持两个值，后面的值无效，例如

```
{between name="id" value="1,3,10"}
输出内容1
{/between}
```

实际判断的范围区间是 `1~3` ，而不是 `1~10` ，也可以支持字符串判断，例如：

```
{between name="id" value="A,Z"}
输出内容1
{/between}
```

name属性可以直接使用系统变量，例如：

```
{between name="Think.post.id" value="1,5"}
输出内容1
{/between}
```

value属性也可以使用变量，例如：

```
{between name="id" value="$range"}
输出内容1
{/between}
```

变量的值可以是字符串或者数组，还可以支持系统变量。

```
{between name="id" value="$Think.get.range"}
输出内容1
{/between}
```

PRESENT/NOTPRESENT标签

present标签用于判断某个变量是否已经定义，用法：

```
{present name="name"}
name已经赋值
{/present}
```

如果判断没有赋值，可以使用：

```
{notpresent name="name"}
name还没有赋值
{/notpresent}
```

可以把上面两个标签合并成为：

```
{present name="name"}
name已经赋值
{else /}
name还没有赋值
{/present}
```

present标签的name属性可以直接使用系统变量，例如：

```
{present name="Think.get.name"}
$_GET['name']已经赋值
{/present}
```

EMPTY/NOTEMPTY 标签

empty标签用于判断某个变量是否为空，用法：

```
{empty name="name"}
name为空值
{/empty}
```

如果判断没有赋值，可以使用：

```
{notempty name="name"}
name不为空
{/notempty}
```

可以把上面两个标签合并成为：

```
{empty name="name"}
name为空
{else /}
name不为空
{/empty}
```

name属性可以直接使用系统变量，例如：

```
{empty name="Think.get.name"}
$_GET['name']为空值
{/empty}
```

DEFINED 标签

DEFINED标签用于判断某个常量是否有定义，用法如下：

```
{defined name="NAME"}
NAME常量已经定义
{/defined}
```

> name属性的值要注意严格大小写

如果判断没有被定义，可以使用：

```
{notdefined name="NAME"}
NAME常量未定义
{/notdefined}
```

可以把上面两个标签合并成为：

```
{defined name="NAME"}
NAME常量已经定义
{else /}
NAME常量未定义
{/defined}
```

资源文件

传统方式的导入外部 `JS` 和 `CSS` 文件的方法是直接在模板文件使用：

```
<script type='text/javascript' src='/static/js/common.js'>
<link rel="stylesheet" type="text/css" href="/static/css/style.css" />
```

系统提供了专门的标签来简化上面的导入：

```
{load href="/static/js/common.js" /}
{load href="/static/css/style.css" /}
```

并且支持同时加载多个资源文件，例如：

```
{load href="/static/js/common.js,/static/css/style.css" /}
```

系统还提供了两个标签别名 `js` 和 `css` 用法和 `load` 一致，例如：

```
{js href="/static/js/common.js" /}
{css href="/static/css/style.css" /}
```

标签嵌套

模板引擎支持标签的多层嵌套功能，可以对标签库的标签指定可以嵌套。

系统内置的标签中，volist、switch、if、elseif、else、foreach、compare（包括所有的比较标签）、（not）present、（not）empty、（not）defined等标签都可以嵌套使用。例如：

```
{volist name="list" id="vo"}
    {volist name="vo['sub']" id="sub"}
        {$sub.name}
    {/volist}
{/volist}
```

上面的标签可以用于输出双重循环。

原生PHP

Php代码可以和标签在模板文件中混合使用，可以在模板文件里面书写任意的PHP语句代码 ，包括下面两种方式：

使用php标签

例如：

```
{php}echo 'Hello,world!';{/php}
```

我们建议需要使用PHP代码的时候尽量采用php标签，因为原生的PHP语法可能会被配置禁用而导致解析错误。

使用原生php代码

```
<?php echo 'Hello,world!'; ?>
```

注意：php标签或者php代码里面就不能再使用标签（包括普通标签和XML标签）了，因此下面的几种方式都是无效的：

```
{php}{eq name='name'value='value'}value{/eq}{/php}
```

Php标签里面使用了 `eq` 标签，因此无效

```
{php}if( {$user} != 'ThinkPHP' ) echo  'ThinkPHP' ;{/php}
```

Php标签里面使用了 `{$user}` 普通标签输出变量 ，因此无效。

```
{php}if( $user.name != 'ThinkPHP' ) echo  'ThinkPHP' ;{/php}
```

Php标签里面使用了 `$user.name` 点语法变量输出 ，因此无效。

> 简而言之，在PHP标签里面不能再使用PHP本身不支持的代码。
>
> 如果设置了 `tpl_deny_php` 参数为true，就不能在模板中使用原生的PHP代码，但是仍然支持PHP标签输出。

定义标签

ASSIGN标签

ASSIGN标签用于在模板文件中定义变量，用法如下：

```
{assign name="var" value="123" /}
```

在运行模板的时候，赋值了一个 `var` 的变量，值是 `123` 。

value属性也支持变量，例如：

```
{assign name="var" value="$val" /}
```

或者直接把系统变量赋值给var变量，例如：

```
{assign name="var" value="$Think.get.name" /}
```

相当于，执行了： `$var = $_GET['name'];`

DEFINE标签

DEFINE标签用于中模板中定义常量，用法如下：

```
{define name="MY_DEFINE_NAME" value="3" /}
```

在运行模板的时候，就会定义一个 `MY_DEFINE_NAME` 的常量。

value属性可以支持变量（包括系统变量），例如：

```
{define name="MY_DEFINE_NAME" value="$name" /}
```

或者

```
{define name="MY_DEFINE_NAME" value="$Think.get.name" /}
```

标签扩展

标签扩展

> 建议开发者将自定义标签库请放置应用目录中，请勿放在框架系统目录内，以免使用 Composer 更新框架时导致自定义标签库的丢失

下面是一个Demo标签库的示例：

```
<?php
namespace app\common\taglib;
use think\template\TagLib;
class Demo extends TagLib{
    /**
     * 定义标签列表
     */
    protected $tags   =  [
        // 标签定义： attr 属性列表 close 是否闭合（0 或者1 默认1） alias 标签别名 lev
el 嵌套层次
        'close'     => ['attr' => 'time,format', 'close' => 0], //闭合标签，默认
为不闭合
        'open'      => ['attr' => 'name,type', 'close' => 1],

    ];

    /**
     * 这是一个闭合标签的简单演示
     */
    public function tagClose($tag)
    {
        $format = empty($tag['format']) ? 'Y-m-d H:i:s' : $tag['format'];
        $time = empty($tag['time']) ? time() : $tag['time'];
        $parse = '<?php ';
        $parse .= 'echo date("' . $format . '",' . $time . ');';
        $parse .= ' ?>';
        return $parse;
    }

    /**
     * 这是一个非闭合标签的简单演示
     */
    public function tagOpen($tag, $content)
    {
        $type = empty($tag['type']) ? 0 : 1; // 这个type目的是为了区分类型，一般来源
是数据库
        $name = $tag['name']; // name是必填项，这里不做判断了
```

```
        $parse = '<?php ';
        $parse .= '$test_arr=[[1,3,5,7,9],[2,4,6,8,10]];'; // 这里是模拟数据
        $parse .= '$__LIST__ = $test_arr[' . $type . '];';
        $parse .= ' ?>';
        $parse .= '{volist name="__LIST__" id="' . $name . '"}';
        $parse .= $content;
        $parse .= '{/volist}';
        return $parse;
    }

}
```

这时候我们的控制器继承 `Controller` ,在配置参数中配置：

```
'template'                => [
            // 模板引擎类型 支持 php think 支持扩展
            'type'          => 'Think',
            // 模板路径
            'view_path'     => '',
            // 模板后缀
            'view_suffix'   => '.html',
            // 预先加载的标签库
            'taglib_pre_load'     =>    'app\common\taglib\Demo',

        ],
```

我们就可以在控制器中对模版赋值：

```
$this->assign('demo_time',time());
```

在模版中调用我们已经预先加载的标签：

```
<h1>开放标签</h1>
{demo:open time='$demo_time'/}
<hr>
<h1>闭合标签</h1>
{demo:close name='demo_name'}
    {$key}=>{$demo_name}<br>
{/demo:close}
<br>
{demo:close name='demo_name' type='1'}
    {$key}=>{$demo_name}<br>
{/demo:close}
```